

Reproduction Instructions

Reproduction Instructions (Self-Hosted, No Lovable)

This guide explains how to take the code in this GitHub repository and stand up the entire Total Event System Solutions platform on your own infrastructure — **without Lovable, without Lovable Cloud, and without the Lovable AI Gateway**. Everything here uses standard open-source tools and third-party services you sign up for directly.

If you instead want to fork the project and keep editing it inside Lovable, see the in-app onboarding flow. This document is for engineers who want to own the deployment end-to-end.

1. What you'll need

Local tooling

- **Node.js 20+** and **Bun 1.1+** (the repo uses Bun for installs and scripts; npm/pnpm work but the lockfile is `bun.lock`).
- **Git**.
- **psql** (PostgreSQL client) or any SQL GUI (TablePlus, DBeaver, pgAdmin).
- **Wrangler CLI** (`npm i -g wrangler`) — Cloudflare's deployment CLI. The repo's `wrangler.jsonc` already targets Cloudflare Workers via `@cloudflare/vite-plugin` .

Third-party accounts (all have free tiers)

Service	Used for	Required?
Supabase (supabase.com)	Postgres, Auth, Storage, RLS	Yes
Cloudflare	Workers hosting (SSR runtime)	Yes (or pick another edge host — see §8)
Resend (resend.com)	Email campaigns	Only if you enable Communications
Stripe (stripe.com)	Ticket payments	Only if you enable ticketing
OpenAI / Anthropic / Google	Image generation (replaces Lovable AI Gateway)	Only if you want auto-generated images
Meta + LinkedIn dev apps	Social Command Center OAuth	Only if you enable that module

2. Clone the repo and install

```
git clone <your-fork-url> total-event-system
cd total-event-system
bun install    # or: npm install / pnpm install
```

The repo is a **TanStack Start v1** app (React 19 + Vite 7). Source layout:

```
src/routes/      File-based routes (TanStack Router)
src/lib/         Server functions (.functions.ts) + helpers
src/integrations/supabase/ Browser, server, and auth-middleware clients
supabase-migrations/ Numbered SQL migrations
```

3. Create the Supabase project

1. Go to supabase.com → **New Project**. Pick a region close to your users. Save the database password somewhere safe.
2. Once the project is provisioned, grab three values from **Project Settings** → **API**:
 - **Project URL** → `SUPABASE_URL`

- **publishable key** (a.k.a. anon key, `sb_publishable_...` or `eyJ...`) → `SUPABASE_PUBLISHABLE_KEY`
- **service_role key** → `SUPABASE_SERVICE_ROLE_KEY` (*server-only, never ship to clients*)

3. Open **Project Settings** → **API** → **Connection string** and copy the `psql` URI.

Wire the publishable values into the repo

The browser client reads its URL/key from `src/integrations/supabase/config.ts`.

Replace the existing values with yours:

```
// src/integrations/supabase/config.ts
export const SUPABASE_URL = "https://<your-ref>.supabase.co";
export const SUPABASE_PUBLISHABLE_KEY = "sb_publishable_...";
```

Commit that change — the publishable key is safe in client bundles because RLS protects every table.

4. Apply the database migrations

All schema lives under `supabase-migrations/` as numbered SQL files. Run them in order, top to bottom:

```
export DATABASE_URL="postgresql://postgres:<password>@db.
  <ref>.supabase.co:5432/postgres"

for f in supabase-migrations/*.sql; do
  echo ">>> $f"
  psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f "$f"
done
```

Latest file as of writing: `038_communications_surveys.sql`.

The migrations are mostly idempotent (`create table if not exists` , etc.) so partial re-runs are safe. They also create RLS policies, GRANTS, storage buckets, and (where used) `pg_cron` / `pg_net` job rows.

Enable the extensions Supabase doesn't enable by default. In the Supabase dashboard → **Database** → **Extensions**, turn on `pg_cron` and `pg_net` (used for scheduled email dispatch). Then re-run any migration that depends on them.

5. Configure storage buckets

The migrations create the buckets the app expects (`department-logos` , `auto-images` , etc.). Confirm they exist under **Storage** → **Buckets** in Supabase. If you skip a migration, the matching bucket will be missing and uploads will silently fail.

6. Set up environment variables

The Cloudflare Worker needs both server-side and client-side variables. Wrangler reads them from `.dev.vars` locally and from `wrangler secret put` in production.

`.dev.vars` for local dev

Create `.dev.vars` at the repo root:

```
# Server-side (process.env.*)
SUPABASE_URL=https://<your-ref>.supabase.co
SUPABASE_PUBLISHABLE_KEY=sb_publishable_...
SUPABASE_SERVICE_ROLE_KEY=eyJ...service-role...

# Email (Communications module)
RESEND_API_KEY=re_...
RESEND_FROM=City Events <hello@yourdomain.com>

# Public site URL (used for unsubscribe links, OG metadata)
SITE_URL=http://localhost:8080

# Cron protection (used by /api/public/dispatch-due)
```

```
DISPATCH_SECRET=<random-32-char-string>

# Stripe (optional)
STRIPE_SECRET_KEY=sk_...
STRIPE_WEBHOOK_SECRET=whsec_...

# Image generation (replaces Lovable AI Gateway – pick ONE)
OPENAI_API_KEY=sk-...
# or ANTHROPIC_API_KEY / GOOGLE_AI_API_KEY – see §9
```

Vite-exposed (browser) variables go in `.env` :

```
VITE_SUPABASE_URL=https://<your-ref>.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_...
VITE_SITE_URL=http://localhost:8080
```

Local dev

```
bun dev      # Vite at http://localhost:8080
```

The Worker runtime is simulated by `@cloudflare/vite-plugin` , so server functions and `/api/public/*` routes work end-to-end locally.

7. Create the first admin user

1. Visit `http://localhost:8080/signup` and create an account with your email.
2. Find the auth user id under Supabase → **Authentication** → **Users**.
3. Promote that user to admin (the platform uses a separate `user_roles` table for security — never store roles on profiles):

```
insert into public.user_roles (user_id, role)
values ('<your-auth-user-id>', 'admin');
```

4. Reload the app. The **Event Ops** sidebar appears at `/staff` .

5. Bootstrap a department at `/staff/admin/departments` , then give yourself a department role at `/staff/admin/permissions` .

8. Deploy to Cloudflare Workers

The repo is preconfigured for Cloudflare Workers (`wrangler.jsonc` → `src/server.ts`).

```
# Log in (browser flow)
wrangler login

# Push every secret your .dev.vars contains
wrangler secret put SUPABASE_URL
wrangler secret put SUPABASE_PUBLISHABLE_KEY
wrangler secret put SUPABASE_SERVICE_ROLE_KEY
wrangler secret put RESEND_API_KEY
wrangler secret put RESEND_FROM
wrangler secret put SITE_URL
wrangler secret put DISPATCH_SECRET
wrangler secret put STRIPE_SECRET_KEY
wrangler secret put STRIPE_WEBHOOK_SECRET
wrangler secret put OPENAI_API_KEY

# Build the SSR bundle, then deploy
bun run build
wrangler deploy
```

Wrangler prints a `*.workers.dev` URL. Point your custom domain at it via Cloudflare DNS, then update `SITE_URL` / `VITE_SITE_URL` to that domain and redeploy.

Other hosts work too. Anything that supports SSR on a V8/Node runtime can host this: Vercel, Netlify, Fly.io, AWS Lambda@Edge, or a plain Node container running `bun run start` . You'll need to adapt the build output and remove `@cloudflare/vite-plugin` from `vite.config.ts` . See the [TanStack Start deployment docs](#) for adapters.

9. Replacing Lovable AI Gateway (image generation)

Some features (auto-generated event hero images, branding artwork) used the **Lovable AI Gateway**. To run without Lovable, swap `src/lib/auto-image.server.ts` to call your provider of choice:

- **OpenAI:** `POST https://api.openai.com/images/generations` with `OPENAI_API_KEY`.
- **Anthropic / Claude vision:** image gen isn't supported; use it only for vision/text.
- **Google Gemini:** `POST https://generativelanguage.googleapis.com/v1beta/models/imggen-3.0:generateImages` with `GOOGLE_AI_API_KEY`.
- **Stability / Replicate / Together:** each has a similar REST endpoint.

The function's contract is: take a prompt, return a PNG buffer, upload it to the `auto-images` Supabase storage bucket, return the public URL. Keep that signature stable and the rest of the app keeps working.

If you don't need auto-generated images, leave the file alone — features that call it will surface a friendly error and the rest of the platform runs fine.

10. Configure optional modules

Communications (email campaigns)

1. Get a Resend API key at resend.com.
2. Verify a sending domain in Resend and set `RESEND_FROM`.
3. Grant `page.communications` to staff under **Admin → Permissions**.
4. For scheduled sends, create a `pg_cron` job in Supabase that hits the public dispatch route every minute:

```
select cron.schedule(  
  'dispatch-due-campaigns',  
  '* * * * *',  
  $$select net.http_post(  
    'https://api.resend.com',  
    {  
      'Authorization': 'Bearer ' || RESEND_API_KEY,  
      'Content-Type': 'application/json',  
      'From': RESEND_FROM,  
      'To': 'staff@lovable.dev',  
      'Subject': 'Your Lovable AI Gateway is ready to go!',  
      'Text': 'Your Lovable AI Gateway is ready to go!',  
      'Html': 'Your Lovable AI Gateway is ready to go!',  
    }  
  )  
);
```

```
url := 'https://YOUR-DOMAIN/api/public/dispatch-due',
headers := jsonb_build_object('x-dispatch-secret', 'YOUR_DISPATCH_SECRET')
)$$
);
```

Surveys

- Migration 038 is all you need. Grant `page.surveys` to staff and you're done. Public anonymous form lives at `/survey/<id>` .

Social Command Center

1. Create Meta and LinkedIn developer apps. OAuth callback URLs:
 - `https://YOUR-DOMAIN/api/public/oauth/meta/callback`
 - `https://YOUR-DOMAIN/api/public/oauth/linkedin/callback`
2. Paste app id / secret at `/staff/admin/social-integrations` .
3. Grant `page.social_command` to staff.

Ticketing & Payments

1. Set `STRIPE_SECRET_KEY` and `STRIPE_WEBHOOK_SECRET` .
2. Add a Stripe webhook endpoint → `https://YOUR-DOMAIN/api/public/stripe-webhook` .

11. Smoke test

After deploying, hit each surface:

- `/` — home page (CMS-driven; seed content at `/staff/admin/home`)
 - `/events` — public event listing
 - `/manual` — full visual user manual (in-app docs)
 - `/staff` — staff dashboard
 - `/staff/admin/permissions` — admin panel
 - `/staff/communications` — campaigns list
 - `/staff/surveys` — surveys list
 - `/survey/<id>` — public anonymous form
-

12. Auth provider notes

The repo's email/password sign-in works out of the box. The OAuth flows shipped in the codebase (Google in particular) were brokered by Lovable Cloud. To run them yourself:

1. In Supabase → **Authentication** → **Providers**, enable Google.
2. Create OAuth credentials in Google Cloud Console; add `https://<your-ref>.supabase.co/auth/v1/callback` as an authorized redirect.
3. Paste the client id/secret into Supabase. The existing `/login` button uses Supabase's `signInWithOAuth({ provider: 'google' })`, which now goes through your own credentials.

Repeat for Apple, GitHub, etc. as needed.

13. Troubleshooting

Failed to resolve import during build — make sure `bun install` finished and that you didn't delete files referenced by the route tree.

new row violates row-level security policy — user isn't signed in, or a migration was skipped. Every public-schema table needs both RLS policies *and* `GRANT` statements; re-run the migration that creates the failing table.

Campaign stuck in scheduled status forever — the `pg_cron` job isn't pinging `/api/public/dispatch-due`. Verify with `select * from cron.job;` and curl the endpoint manually to confirm `DISPATCH_SECRET` matches.

Email comes from `onboarding@resend.dev` — `RESEND_FROM` isn't set. That address is Resend's shared test sender (max 100/day).

SSR error referencing `LOVABLE_API_KEY` or `lovable-ai-gateway` — you're hitting a code path that still calls the gateway. Swap that helper to your own provider as in §9.

Worker deploy fails with "Module not found: cloudflare:workers" — you're running on Node directly. Either run via Wrangler (`wrangler dev`) or switch hosts

and remove `@cloudflare/vite-plugin` from `vite.config.ts` .

`__dirname is not defined at runtime` — a Node-only npm package slipped into a server function. Replace it with a Web-standard or fetch-based alternative; the Worker runtime can't shim it.

14. Going further

- **Backups:** Supabase runs daily backups on paid tiers. For self-managed Postgres, `pg_dump` on a schedule.
- **Custom domain:** Cloudflare DNS → CNAME → your worker. Update `SITE_URL` and Supabase redirect URLs afterwards.
- **Monitoring:** Workers Analytics + Supabase Logs cover most of it; pipe to Sentry / Logflare if you want richer traces.
- **CI/CD:** The repo has no opinionated pipeline. A 20-line GitHub Action that runs `bun install && bun run build && wrangler deploy` on push to `main` is enough.

You now own every layer of the stack — code, database, runtime, secrets. Lovable is no longer in the loop.